

UniGames: MongoDB e GraphQL

Roteiro de apresentação • José Artur • Sistemas de Informação

1. Como explicar a arquitetura

O UniGames tem três partes: a interface em React, o backend em Node.js e o banco MongoDB. A interface recebe o que o usuário faz nas telas. O backend valida os dados, confere as permissões e realiza as consultas ou gravações. O MongoDB guarda os documentos.

O arquivo enviado já tinha conexão com o MongoDB Atlas, usando o banco unigames. Também havia schema e resolvers GraphQL. Porém, o front estava configurado com `VITE_DATA_SOURCE=demo`: nesse modo, as telas usavam dados locais do navegador.

Na versão revisada, o arquivo de configuração de exemplo usa GraphQL. A conexão continua sendo feita pelo Mongoose, e o servidor aguarda o banco antes de abrir.

Parte	O que faz no projeto
React	Mostra as páginas e recebe os dados dos formulários
Apollo Client	Envia queries e mutations para o backend
Express + Apollo Server	Recebem as requisições em <code>/graphql</code>
Services + models do Mongoose	Aplicam as regras e consultam ou gravam documentos
MongoDB	Armazena usuários, jogos, ofertas, anúncios, avaliações e relatórios

Uma explicação curta para começar:

“A tela não acessa o MongoDB diretamente. Ela manda uma operação GraphQL para o backend. O backend valida essa operação, aplica as regras do sistema e usa o Mongoose para ler ou gravar os dados no MongoDB.”

2. Onde acontece a conexão com o banco

Passo 1 — configurar o ambiente

O backend lê o arquivo `backend/.env`. A variável `MONGODB_URI` informa o endereço do banco. No Atlas, ela contém o cluster, o usuário do banco, sua senha e o nome do banco. A variável `JWT_SECRET` é usada para assinar os tokens de login.

Exemplo com valores que precisam ser substituídos:

```
PORT=3000
MONGODB_URI=mongodb+srv://USUARIO:SENHA@SEU_CLUSTER.mongodb.net/unigames
JWT_SECRET=SUA_CHAVE_PROPRIA
```

No Atlas, é preciso ter um usuário do banco e permitir o IP do computador que executa o backend. A conta de acesso ao site do Atlas e o usuário do banco são coisas diferentes. Copie a URI pela opção de conexão para drivers; preserve também as opções de conexão fornecidas pelo Atlas.

O `.env.example` da entrega usa um MongoDB local. Para continuar com o Atlas do grupo, copie sua URI para o `.env` do backend. Se a senha tiver caracteres especiais, use a forma codificada exigida pela URI.

Passo 2 — carregar as variáveis

Em `backend/src/server.js`, o `dotenv` carrega o arquivo pelo caminho da pasta backend. Assim, as informações ficam disponíveis em `process.env`.

Passo 3 — abrir a conexão

Em `backend/src/database.js`, a função `conectarBanco` usa:

```
await mongoose.connect(uri, {
  serverSelectionTimeoutMS: 10000,
});
```

O `await` aguarda o resultado. O servidor chama `await conectarBanco()` antes de abrir a porta HTTP. Se a conexão falhar, ele mostra o erro e não inicia como se o banco estivesse disponível.

O Mongoose mantém a conexão compartilhada pelos models. Cada model não precisa abrir uma conexão separada. A variável opcional `MONGODB_DNS_SERVERS` permite configurar DNS caso a rede apresente o erro de consulta SRV.

Para explicar:

“O `dotenv` lê a configuração, a função `conectarBanco` abre a conexão com `mongoose.connect` e o servidor só começa a atender depois que essa conexão é estabelecida.”

3. Da tela até o documento: criar um anúncio

Este é um exemplo completo para demonstrar à professora.

1 — o formulário monta os dados

A tela `frontend/src/pages/FormAnuncio.jsx` reúne o jogo escolhido, preço, plataforma, conservação, descrição e contato. Ao enviar, chama `api.saveListing(id, input)`. Para um anúncio novo, não há um id de anúncio.

2 — o serviço escolhe GraphQL

`frontend/src/services/api.js` escolhe o adaptador a partir de `VITE_DATA_SOURCE`. Com o valor `graphql`, usa o serviço de `frontend/src/services/graphql.js`.

Esse serviço executa `CREATE_LISTING`, declarada em `frontend/src/graphql/mutations.js`. A operação tem este formato:

```
mutation CriarAnuncio($input: CreateListingInput!) {
  createListing(input: $input) {
    id
    preco
    status
    jogo { id titulo }
    vendedor { id nome }
  }
}
```

Os dados são enviados como variáveis, separados do texto da operação.

`frontend/src/services/apollo.js` configura o endereço HTTP e acrescenta o token de login ao cabeçalho `Authorization`.

3 — o backend recebe e resolve a operação

O Apollo Server recebe a requisição em `/graphql`. O schema valida o nome da operação, os tipos de entrada e os campos pedidos.

`backend/src/graphql/resolvers/revendaResolvers.js` encaminha `createListing` para `domainService.createListing(input, usuario)`.

4 — as regras são conferidas e o model grava

O serviço verifica se existe uma sessão válida, se o usuário pode anunciar, se o jogo existe e se o preço é válido. O vendedor é obtido da sessão autenticada.

A gravação usa o model `Listing`:

```
const row = await Listing.create({
  ...listingFields(input),
  jogo: input.jogoId,
  vendedor: user.id,
  status: 'ativo',
  vendidoEm: null,
});
```

O Mongoose valida os campos do schema e envia a inserção ao MongoDB. O documento fica na coleção de anúncios, com seu identificador e timestamps.

5 — os dados voltam para a tela

O serviço busca o anúncio com jogo e vendedor preenchidos. O GraphQL retorna somente os campos solicitados. O front monta uma instância da classe `Listing` de `entities.js`, atualiza os dados da interface e volta para “Meus anúncios”.

4. Usuários, autenticação e relações

Cadastro e login

O cadastro passa por `backend/src/services/userService.js`. O nome é normalizado, o e-mail é convertido para minúsculas e a senha precisa ter pelo menos seis caracteres. Antes de salvar, o `bcrypt` gera um hash da senha.

Hash não é uma senha que o sistema pode “descriptografar”. No login, `bcrypt.compare` compara a senha digitada com o hash armazenado.

Depois do cadastro ou login, o backend gera um JWT com o id do usuário. O front guarda a sessão em `sessionStorage` e envia o token nas requisições seguintes.

Como o backend sabe quem está usando

O backend lê o cabeçalho `Authorization: Bearer TOKEN`, valida a assinatura e consulta o usuário no banco. O resultado é colocado em `context.usuario`, disponível aos resolvers GraphQL.

Essa consulta permite usar o perfil atual da conta. Um token de uma conta excluída não continua autorizando operações.

A ativação de revenda usa `tornarRevendedor`. Ela valida a data de nascimento, confere a idade mínima de 16 anos e salva a mudança. A tela “Meus anúncios” usa esse serviço e atualiza a sessão sem recarregar a página.

Como os documentos se relacionam

MongoDB usa coleções de documentos. Neste projeto, algumas relações são armazenadas com `ObjectId` e a opção `ref` dos schemas do Mongoose.

Campo	Referência
<code>Listing.jogo</code>	Documento de Game
<code>Listing.vendedor</code>	Documento de User
<code>PriceOffer.jogo</code>	Documento de Game
<code>Review.avaliador</code>	Usuário que escreveu a avaliação
<code>Review.jogo</code> ou <code>Review.avaliadoUser</code>	Um único alvo da avaliação

O método `populate` busca os documentos referenciados e preenche os dados que serão exibidos. Por isso a tela pode receber o título do jogo e o nome do vendedor, em vez de mostrar apenas os ids.

As respostas públicas usam `PublicUser`, sem e-mail e data de nascimento. Os dados completos da conta ficam no próprio perfil ou nas operações de admin.

5. O que está definido no GraphQL

No GraphQL, os termos mais precisos são “tipos”, “inputs” e “resolvers”. As classes de JavaScript usadas pelas telas estão em `entities.js`. Os schemas do Mongoose e o schema do GraphQL têm papéis diferentes: o primeiro define o documento do banco; o segundo define o contrato da API.

Entidade	Operações principais
User	Cadastro, login, meu perfil, perfil público, edição, ativação de revenda, listagem administrativa e exclusão
Game	Consulta, pesquisa, filtros, criação, edição e exclusão
Listing	Anúncios públicos e pessoais, detalhes, criação, edição e exclusão
PriceOffer	Ofertas, destaques, histórico, criação, atualização e exclusão
Review	Consulta por jogo ou usuário, criação, edição e exclusão
Report	Geração, consulta e exclusão de relatórios salvos

`backend/src/graphql/typeDefs.js` declara esses contratos. Cada entidade tem um arquivo em `backend/src/graphql/resolvers/`. O arquivo `resolvers.js` reúne as operações e `schema.js` entrega o conjunto ao Apollo Server.

Query é usada para consultar dados. Mutation é usada para operações que alteram dados. O schema permite pedir apenas os campos necessários:

```
query ConsultarJogos {
  getGames {
    id
    titulo
    plataformas
  }
}
```

Relatórios e histórico

O relatório pessoal `getMyReport` é calculado a partir dos anúncios da conta. Ele mostra o total, os ativos, os vendidos e a soma dos valores declarados.

`generateReport` salva um relatório administrativo na coleção de Report. Os tipos disponíveis para geração são atividade de usuários, ofertas em destaque e variação dos preços registrados. Esses relatórios guardam um retrato dos dados no momento da geração.

O projeto ainda não registra pesquisas para gerar “buscas frequentes”. Também não coleta preços automaticamente em lojas externas. As novas operações estão na API e no adaptador GraphQL; nem todas têm uma tela adicional dedicada.

Datas e horários usam `DateTime` em ISO 8601. A data de nascimento é retornada como AAAA-MM-DD. O scalar `JSON` representa os dados consolidados do relatório.

6. Um roteiro para a apresentação

Uma fala possível, para adaptar ao seu jeito:

“A integração foi feita pelo backend. A gente configurou a URI do MongoDB no arquivo de ambiente e usou o Mongoose para abrir a conexão. Os models representam os dados que serão salvos. Quando alguém usa uma tela, o React chama o serviço GraphQL, que envia uma query ou mutation ao Apollo Server. O resolver encaminha a operação para o serviço responsável. Esse serviço valida os dados e as permissões e usa o model para consultar ou gravar no MongoDB. Depois o resultado volta para a tela.”

Para demonstrar, faça esta sequência em um banco configurado para apresentação:

1. Mostre `database.js` e a chamada `await conectarBanco()` no servidor.
2. Mostre `models/Listing.js` e suas referências para jogo e vendedor.
3. Mostre `typeDefs.js` e `revendaResolvers.js`.
4. Cadastre uma conta, faça login e ative a revenda.
5. Crie um anúncio de um jogo existente no catálogo.
6. Confira o documento no MongoDB Compass ou no painel de dados do Atlas.
7. Edite o anúncio, marque como vendido e consulte o relatório pessoal.

Configure `VITE_DATA_SOURCE=graphql` antes da demonstração e reinicie o Vite. O modo demo usa o armazenamento do navegador; seus dados não aparecem no Atlas.

Se o banco estiver vazio, o seed opcional acrescenta jogos. Ele não cria ofertas de preço fictícias. O README explica como configurar o catálogo e um administrador.

Perguntas que a professora pode fazer

“Qual arquivo faz a conexão?” `backend/src/database.js`. O servidor carrega o ambiente e chama essa função.

“O GraphQL guarda os dados?” Não. Ele define e executa as operações da API. Os dados persistem no MongoDB, acessado pelos models do Mongoose.

“O front sabe a senha do banco?” Não. Ele conhece a URL da API. A URI e as credenciais do banco ficam no backend.

“Quem pode mudar um anúncio?” Seu proprietário ou um administrador. O backend confere isso pela sessão.

“Por que existem REST e GraphQL?” São duas interfaces do backend. O projeto consegue usar qualquer uma, compartilhando as regras e o mesmo banco.

“O relatório comprova que uma venda foi paga?” Não. Ele soma anúncios que o usuário marcou como vendidos. Não existe processamento de pagamento.

Referências oficiais para consulta

- Mongoose: <https://mongoosejs.com/docs/connections.html>
- Relações e populate: <https://mongoosejs.com/docs/populate.html>
- MongoDB Atlas: <https://www.mongodb.com/docs/atlas/connect-to-database-deployment/>
- Apollo Server e Express: <https://www.apollographql.com/docs/apollo-server/api/express-middleware>
- Resolvers: <https://www.apollographql.com/docs/apollo-server/data/resolvers>
- Token no Apollo Client: <https://www.apollographql.com/docs/react/networking/authentication>